American University of Sharjah

CMP 466: Machine Learning_Data Mining

Dr. Salam Dou

Nov 21, 2025

Project

Interpretable Fitness Prediction with Classical ML models

Mostafa Ahmed b00092727 (sections in green)

Ibrahim Abdallah b00096538 (sections in red)

Abstract

We've decided to investigate heart disease as our project for this course, as it is the leading global cause for death, claiming 32% of deaths worldwide [1] and at the same time today's technology has allowed us to collect metrics about people's lifestyle that may affect the probability of them developing heart disease (blood pressure, heart rate, weight, sleep duration, activity, nutrition). Turning these raw numbers into a simple, reliable indication of whether someone is in a “fit” or “not fit” category is not something doctors can always do by eye, especially when several variables interact at the same time.

In this project we use supervised machine learning to predict an individual’s fitness status based on a small set of easily collected variables: age, anthropometric measurements, cardiovascular indicators, sleep, and lifestyle. The target variable is a binary label which indicates whether the person can be considered physically fit according to the dataset’s original definition.

Classical machine-learning algorithms such as KNN, decision trees, and Naive Bayes are especially suitable for this kind of health data. They are relatively simple, fast to train, and their behavior can often be interpreted in terms of distances between patients, decision rules on specific features, or probabilistic assumptions on the data distribution.

A lot of earlier work in this area focuses on disease-specific prediction (for example, predicting heart disease) or on models trained on high-frequency wearable-sensor signals rather than simple summary statistics. These models are often accurate but can be harder to interpret and may depend on data that is not always available in everyday clinical practice. In contrast, our project deliberately works with a compact feature set that could, in principle, be collected during a routine checkup or via a simple questionnaire.

We used a dataset of 70,000 records with 12 variables (excluding ID), combining demographic, physiological, and lifestyle information into one table perfect for supervised learning.

1 - Introduction

Using data to understand someone's cardiovascular fitness has always made sense, because fitness depends on many things working together. Age, sleep, activity, weight, and basic vital signs all play a role, and it's not always obvious how they interact. That is why so many studies have tried using machine learning for heart disease and fitness prediction. But a lot of those attempts struggled with the same basic issues: messy data, missing values, inconsistent categories, and features that are all on different scales. When that happens, even simple models end up not producing reliable results.

In our project, we worked with a fitness classification dataset instead of a medical heart disease dataset, but the challenges are similar. If the data is not cleaned and prepared properly, the models will give misleading results. To fix this issue, we heavily preprocessed the data before going ahead with any of the supervised learning and testing models. We encoded variables that seemed to be categorical on paper, but could practically be turned into numerical values, while eliminating unnecessary junk like superfluous cholesterol values. We also normalized the measurements so the distance-based methods wouldn't get biased towards variables that naturally have higher values.,

With these steps in place, simple models like knn, decision trees, and naive bayes can perform more reliably and actually show what they're capable of when the data is prepared the right way.

2 - Literature Review

Cardiovascular risk and fitness-related prediction research is growing rapidly, as structured health datasets make it easier for even simple models to uncover patterns related to age, lifestyle, and physiological signals. While many studies use classical machine learning techniques similar to those in our project, they often run into common issues like incomplete/incorrect preprocessing, unbalanced datasets, and a lack of fair comparisons across models. The eight papers summarized below illustrate different approaches, their contributions, and their limitations.

[2] Application of Naive Bayes Algorithm for Physical Fitness Level Classification

This paper examines the use of Naïve Bayes algorithms to classify individuals into fitness-level categories based on basic physiological and lifestyle variables, which is an approach that closely mirrors our own setup. The authors primarily employ gaussian naive bayes and evaluate its effectiveness on a small, non-public dataset collected in a controlled fitness assessment environment. Key metrics include accuracy, precision, recall, and confusion matrices, with reported accuracies consistently falling in the 70–80% range, depending on the feature set.

[3] MHfit: Mobile Health Data for Predicting Athletics Fitness Using Machine Learning

MHfit investigates athletic fitness prediction using mobile health data like step count, sleep patterns, and activity duration, all attributes that resemble those in our dataset. The study evaluates models including logistic regression, Naive Bayes, decision trees, random forests, and XGBoost, benchmarking performance using accuracy, F1-score, and ROC curves. Tree-based

and ensemble models clearly outperform Naïve Bayes and KNN, achieving accuracies between 85–90%.

Though MHfit uses time-sensitive mobile sensor data (unlike our static dataset), the shared goal of mapping everyday health metrics to fitness levels remains highly relevant. The authors highlight limitations such as device-dependence and data noise, while our data is cleaner but less detailed. Their success with non-linear models supports our decision to include tree-based methods and explains why such models can surpass simpler alternatives in health prediction tasks.

[4] Machine Learning–Based Fitness Tracking Analysis

This study evaluates ML models for classifying physical activity levels (low, moderate, high) using features like step count, calories burned, sleep duration, and demographics, closely aligned with our dataset. The authors test D Ts, logistic regression, Naive Bayes, and SVM, reporting performance with accuracy, F1-score, and confusion matrices. Decision Trees and SVM outperform the rest with accuracies around 80–85%.

The authors stress the importance of clean preprocessing. Inconsistent categorical variables and unscaled numerical features severely reduce the performance of models like KNN and Naïve Bayes. They also emphasize the need for class balance, which is a factor we address using stratified splitting. Ultimately, this paper reinforces that with clean and well-scaled data, simple models can yield solid performance, supporting our own preprocessing pipeline.

[5] A Comparative Analysis of Machine Learning Models for Obesity Prediction

While focused on obesity prediction rather than general fitness, this paper uses a dataset nearly identical to ours, composed of demographics, physical measurements, and behavioral variables with a binary label. It compares models like KNN, decision trees, Naive Bayes, Random Forests, SVM, and Logistic Regression, with accuracy, precision, recall, and F1-score ranging between 70–90%.

The study highlights how different models respond to noisy or correlated lifestyle variables. KNN suffers without feature scaling, Decision Trees risk overfitting without tuning, and Naive Bayes offers steady but mid-tier results. These patterns match our findings, particularly the need to tune Decision Tree depth, KNN distance metrics, and Naïve Bayes smoothing. The trade-off between model accuracy and interpretability, especially for health-related use, is also a key consideration discussed.

[6] Implementation of K-Nearest Neighbors, Naïve Bayes, Decision Tree, and SVM for Obesity Prediction

This comparative study focuses on the same four models used in our project: KNN, NB, DT, and SVM. The structured health dataset, built from clinical questionnaires, closely resembles ours. Performance is measured via accuracy and confusion matrices, with Decision Trees slightly outperforming KNN and Naïve Bayes (mid-70s to low-80s accuracy), while SVM typically performs best.

The paper emphasizes hyperparameter tuning, showing that tuning significantly boosts accuracy, for example, adjusting KNN neighbors or DT depth improves results by 5–10%. This mirrors our own project design, where tuning is mandatory. Despite a small dataset, the methodological insights around tuning and comparative evaluation are directly relevant to our model-selection strategy.

[7] Naïve Bayes, Decision Tree, and K-Nearest Neighbor for Heart Disease Prediction

This study uses the same three core models as our project, KNN, decision trees, and Naive Bayes, all to predict heart disease using the UCI Heart Disease dataset. Features include age, cholesterol, blood pressure, and chest pain type. Performance is reported via accuracy, sensitivity, specificity, and confusion matrices. Decision Trees generally lead (78–82% accuracy), with KNN and Naive Bayes slightly behind.

A key contribution is the impact of feature scaling on KNN. Without it, KNN performs poorly; with standardization, it becomes competitive. The study also notes that Naive Bayes tends to underpredict minority classes, which is a concern relevant to our moderately imbalanced dataset. Their preprocessing strategies reinforce many of our decisions, including standardizing numerical values and encoding categorical features carefully.

[8] Heart Attack Prediction Using Machine Learning: A Comparative Study of Naive Bayes, Decision Tree, Random Forest, and K-Nearest Neighbors

This comparative study assesses traditional ML models for heart attack prediction using structured clinical data. Metrics include accuracy, sensitivity, specificity, and ROC curves. Random Forest often achieves the best results (above 85%), while decision trees and KNN also perform competitively, similar to classroom datasets.

A standout point is the benefit of combining feature selection (e.g., SelectKBest) with PCA. These techniques improve KNN accuracy and help stabilize Naïve Bayes by removing noise and multicollinearity. Our own experiments show similar improvements. The paper also emphasizes the importance of model interpretability in clinical applications—reinforcing our focus on transparent models.

[9] Risk Assessment of Cardiovascular Diseases Using kNN and Decision Tree

This paper evaluates KNN and Decision Trees for cardiovascular risk prediction using a dataset of vitals and lifestyle factors—nearly identical in structure to ours. Accuracy, confusion matrices, and ROC curves are reported. Again, Decision Trees outperform KNN (80–85% accuracy), though KNN can perform well with careful tuning.

A finding is the exploration of distance metrics for KNN: Manhattan distance occasionally outperforms Euclidean on mixed medical features. This justifies our own experiments with distance-metric tuning. While the dataset is small, the methodological points remain strong and validate our strategy of evaluating both distance- and tree-based models under proper scaling and tuning.

Overall Summary
Across all the papers, one thing shows up again and again: simple, classical models like KNN, decision trees, and Naive Bayes can actually do a solid job when the data is structured and easy to collect. Most studies report that Decision Trees usually come out on top, while KNN and Naive Bayes work well as long as the data is cleaned, encoded, and scaled properly. At the same time, a lot of the existing work relies on private datasets that are not really practical for everyday use. Some papers also spend very little time on the preprocessing steps, even though those steps end up making the biggest difference for these models. This is what makes our approach stand out. We focus on a simple and accessible dataset, apply a clear and consistent preprocessing process, and compare several classical models.

3. Methods and Datasets

3.1 Dataset

The dataset used in this project is a structured cardiovascular and lifestyle dataset containing 70,000 individuals, with 12 (11 excluding ID) features:

Id, age, gender, height, weight, ap_hi, ap_lo, cholesterol, gluc, smok, alco, active.

The class feature is *cardio,* a binary value representing whether the individual suffers from heart disease.

The data is clean overall, with only no columns having missing values, and no duplicates.

The class distribution is very close between healthy and affected individuals, almost 50/50.

It well-suited for the kinds of classical machine-learning models used in this project.

3.2 Data Preprocessing
Our preprocessing started with loading the raw cardiovascular dataset and doing a quick inspection to understand what we were working with. We checked the shape of the data, the column types, basic statistics and if there were any missing values or duplicates (fortuanately there weren't any). created a clearer, more practical age feature by converting the original age in days into age in years, since it is easier to interpret and more meaningful. We also looked at the target class distribution to see whether the dataset was balanced, and then explored the correlations between the numerical features to get a sense of how they relate to each other. Outliers in measurements like height, weight and blood pressure were flagged using the IQR method, although we did not remove them so that the modeling stage could decide how to handle them. Since several features were numerical labels rather than true continuous values we used one hot encoding to turn columns like gender, cholesterol level, glucose level into proper categorical variables that would better suit the models. After encoding, we scaled the continuous features with StandardScaler so they all share the same mean and standard deviation, and we scaled the remaining features with MinMaxScaler to keep everything in a consistent numeric range. Finally, we separated the target from the features and performed a stratified train test split so that the class balance would remain stable across both sets. This left us with a clean and fully processed dataset that is ready for modeling.

3.3 Machine Learning Algorithms

In this project, we used three classification algorithms, KNN (k-Nearest Neighbors), Decision Tree, and Naive Bayes for our dataset to predict whether a patient has cardiovascular disease. We will discuss each ML algorithm that we used:

- KNN is one of the simplest algorithms. The idea is very straightforward, in our dataset when we want to classify a new patient, we look at the k most similar patients in the training data and see which class they belong to. Then the new patient is assigned to the majority class of their neighbours. The similarity is measured through a distance metric such as Manhattan distance: $d(x, x') = \sum |xi - xi'|$

  This tells us how close two points are, but in this case how the new patient is close to other patients. In some cases, we might give a weight to the ones that are closer to the new patient. KNN does not create a model in advance, hence it gets the name "lazy learner". It works well when data is not too noisy, meaning no outliers which can make data noisy. Also, it can be affected by irrelevant features, so scaling and feature selection are important.

  In our project, we tested different KNNs by adjusting the parameters such as the k, distance, and weight. We also used it as a baseline method to see how it performs with our dataset without hyperparameter tuning.

- Decision Tree is a model that makes predictions by asking a series of questions about the data, for example, Do you exercise?, etc. Each question splits the data into groups, and the tree keeps splitting until it reaches a final decision often called the "leaf". Decision Trees try to find splits that create purer subsets, often using impurity measures such as the Gini index: $Gini = 1 - \sum pi^2$

  Depending on the answers, the tree will move right or left until it reaches the target class, which in our cases is disease or no disease. This model is popular since it is easy to understand and follow because it is based on basic rules. However, a drawback is that if the tree grows too deep, it may start memorizing the training data which is overfitting. To avoid that, we must tune the parameters to get the most optimal to avoid overfitting.

  In our project, we tried different settings for these parameters one at a time, as required from the document and then selected the best versions of each parameter.

- Naive Bayes is a very fast and simple model which is based on probability: $P(Y|X) = P(X|Y)P(Y)/P(X)$

  It uses the well known theorem the Bayes' Theorem, which combines likelihood of each feature with the probability of the overall class. As you may guess, it is called naive since it considers all features are independent from each other, which is obviously not always true. Take our dataset for example, blood pressure and age can be related. Even though it does not consider some features dependent, it works pretty well. There are different types of feature types such as Gaussian BN, which assumes all features follow a normal distribution, which is why it works well with continuous values. Bernoulli NB works with binary features and Multinomial NB is used for count based features. Also Bernoulli and Multinomial NB work with negative values, hence we use MinMaxScaler for it.

  In our project, we tested all three versions and found that Gaussian BN performed the best, which was not surprising since our dataset is mostly continuous numeric features.

3.4 Feature Selection and Dimensionality Reduction:

- Feature Selection is used to pick only the most useful features for prediction. The goal is to remove noise, reduce overfitting, and make models easier and faster to train. Some models, like KNN, benefit a lot from FS because they rely mainly on distances, and the irrelevant features can confuse the algorithm, which makes FS an improvement for it.

  In our project, we used a filter method called Mutual Information (MI). MI measures how much a feature tells us about the target label. A higher MI score means the feature is more informative.

  We calculated the MI score for every feature and ranked them from most to least important. Then we tested for each model different numbers of top features to see how performance changed.

- Dimensionality reduction aims to reduce the number of features by creating new ones that still capture most of the important information in the data. In our project, we used Principal Component Analysis (PCA), which is an unsupervised method that transforms the original features into a new set of components.

  PCA creates principal components that are not correlated with each other. The first component captures the most variance, the second captures the next most, and so on. This helps remove linear correlations between features and compress the data while keeping most of its overall structure.

  In our project, we applied PCA to the standardized dataset and looked at the cumulative explained variance to decide how many components to keep. We selected the number of components that preserved at least 90% of the variance. Later in the results section, these components were used as inputs to our models to see how a lower-dimensional representation affects performance.

3.5 Model Selection:

To train and choose our models, we used an 80 - 20 train test split together with 5-fold stratified cross-validation (CV) on the training data. We first separated 20 percent of the dataset as a hold-out test set, which we did not use during training or tuning. The remaining 80 percent was used for model development.

Inside the training set, we applied a 5-fold CV to select the best hyperparameters for each algorithm. Which divides the training data into five equal parts. The model is trained on four parts and validated on the remaining one, and this process repeats until every fold has been used once for validation.

We used this cross-validation procedure while tuning the main hyperparameters for KNN, Decision Tree, and Naive Bayes. After identifying the best settings, we retrained each final model on the full 80 percent training set and evaluated it once on the 20 percent test set.

3.6 Evaluation Metrics:

To evaluate the performance of our classification models, we used several metrics. Each one highlights a different aspect and together they give a complete picture of how well the models performed.

- Accuracy measures the proportion of correct predictions out of all predictions. It is calculated as: $Accuracy = TP + TN / TP + TN + FP + FN$. Which gives a quick overall sense of performance, but by itself it may be misleading, especially if the classes are imbalanced or certain types of errors matter more than others.
- Precision measures how many of the patients predicted as positive (cardio = 1) truly have the disease. It is calculated as: $Precision = TP / TP + FP$. High precision means the model makes fewer false alarms.
- Recall measures how many of the actual positive cases were correctly identified. It is calculated as: $Recall = TP / TP + FN$. In medical applications, recall is especially important because false negatives can be critical.
- F1-score is the harmonic mean of precision and recall. It is given by: $F1 = 2 \times (Precision * Recall) / (Precision + Recall)$. It balances both types of errors, making it a more informative measure than accuracy. In our project, we used the weighted F1-score during cross-validation.
- Finally, we used the confusion matrix, which shows the counts of true positives, true negatives, false positives, and false negatives. This allowed us to directly inspect the types of errors each model makes and understand whether the model tends to miss disease cases or incorrectly flag healthy patients.
- The ROC-AUC score shows how well a model can tell the difference between the two classes at different thresholds. The ROC curve plots how many real positives it catches (recall) versus how many negatives it mistakes as positives (false positives). The AUC ranges from 0.5 (no skill) to 1.0 (perfect). It's helpful because it doesn't rely on a single cutoff point to evaluate the model.

4 - Results

4.1 - Data preprocessing

Our preprocessing started with loading the raw dataset and doing a quick inspection to understand what we were working with. We checked the shape of the data, the column types, basic statistics and if there were any missing values, wich there weren't

```
shape before anything: (70000, 13)
```

	id	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	cardio
0	0	18393	2	168	62.0	110	80	1	1	0	0	1	0
1	1	20228	1	156	85.0	140	90	3	1	0	0	1	1
2	2	18857	1	165	64.0	130	70	3	1	0	0	0	1
3	3	17623	2	169	82.0	150	100	1	1	0	0	1	1
4	4	17474	1	156	56.0	100	60	1	1	0	0	0	0

After that, we checked if there were any duplicate rows, which there were also none of, and added a clearer age feature by converting the original age in days into age in years, since it is easier to interpret and more meaningful.

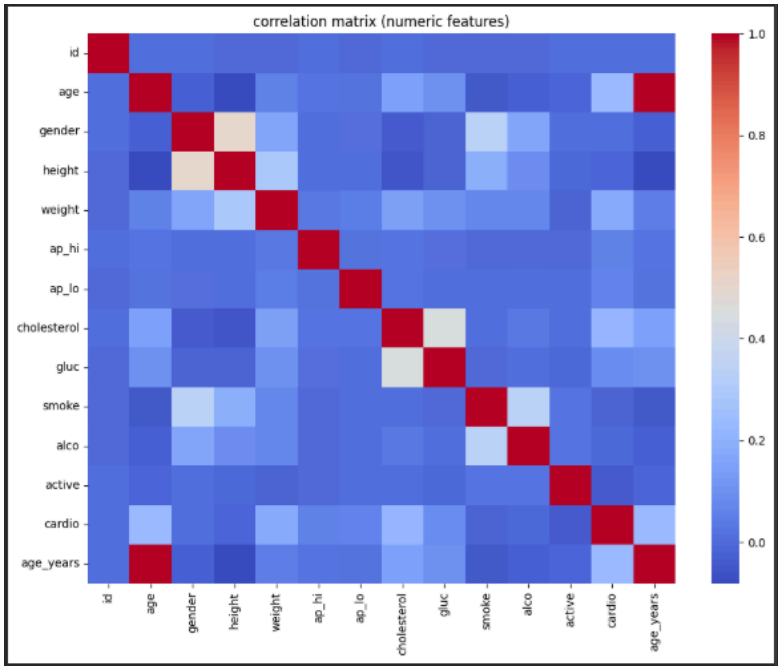
```
duplicates removed: 0

preview after cleaning:
```

	id	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	cardio	age_years
0	0	18393	2	168	62.0	110	80	1	1	0	0	1	0	50.4
1	1	20228	1	156	85.0	140	90	3	1	0	0	1	1	55.4
2	2	18857	1	165	64.0	130	70	3	1	0	0	0	1	51.7
3	3	17623	2	169	82.0	150	100	1	1	0	0	1	1	48.3
4	4	17474	1	156	56.0	100	60	1	1	0	0	0	0	47.9

We also looked at the target class distribution to see whether the dataset was balanced, and then explored the correlations between the numerical features to get a sense of how they relate to each other, and normalized the features so that the models are not biased to features with naturally higher values.

```
scaling done. sample rows:
```

	age	height	weight	ap_hi	ap_lo	cardio	age_years	gen
0	0.588076	0.443452	-0.847873	-0.122182	-0.088238	0	-0.434868	
1	0.730159	-1.018168	0.749831	0.072610	-0.035180	1	0.304833	
2	0.624003	0.078047	-0.708942	0.007679	-0.141297	1	-0.242546	
3	0.528455	0.565254	0.541435	0.137541	0.017879	1	-0.745543	
4	0.516918	-1.018168	-1.264666	-0.187113	-0.194356	0	-0.804719	



Outliers in measurements like height, weight and blood pressure were flagged using the IQR method, although we did not remove them so that the modeling stage could decide how to handle them.

	feature	num_outliers	lower_bound	upper_bound
0	age_years	4	33.4	73.4
1	height	519	142.5	186.5
2	weight	1819	39.5	107.5
3	ap_hi	1435	90.0	170.0
4	ap_lo	4632	65.0	105.0

Since several features were numerical labels rather than true continuous values we used one hot encoding to turn columns like gender, cholesterol level, glucose level into proper categorical variables.

```python
categorical_cols = ["gender", "cholesterol", "gluc", "smoke", "alco", "active"]

data_no_id = data.drop(columns=["id"])

data_encoded = pd.get_dummies(
    data_no_id,
    columns=categorical_cols,
    drop_first=True
)

print("shape after one-hot encoding:", data_encoded.shape)
print("new columns:")
print(data_encoded.columns.tolist())


target = data_encoded["cardio"].copy()

le = LabelEncoder()
y = le.fit_transform(target)

print("unique labels after label encoding:", np.unique(y))
```

```
shape after one-hot encoding: (70000, 15)
new columns:
['age', 'height', 'weight', 'ap_hi', 'ap_lo', 'cardio', 'age_years', 'gender_2', 'cholesterol_2', 'cholesterol_3', 'gluc_2
```

Finally, we separated the target from the features and performed a stratified train test split so that the class balance would remain stable across both sets. This left us with a clean and fully processed dataset that is ready for modeling.

```
final feature matrix shape: (70000, 14)
final target shape: (70000,)


X_train: (56000, 14)
X_test: (14000, 14)
y_train: (56000,)
y_test: (14000,)

class distribution in full data:
cardio
0    0.5003
1    0.4997
Name: proportion, dtype: float64

class distribution in training set:
cardio
0    0.500304
1    0.499696
Name: proportion, dtype: float64

class distribution in test set:
cardio
0    0.500286
1    0.499714
Name: proportion, dtype: float64
```

4.2 Machine Learning Results:

Before starting any tuning or feature selection, we evaluated each of our three models (KNN, Decision Tree, and Naive Bayes) in their baseline level, meaning the default settings. This gave us a starting point to understand how the models behave "out of the box" with no preprocessing beyond scaling. We refer to these as our baseline models.

This step is important because it helps us know what improvement we gain later after tuning hyperparameters and selecting features. For KNN, we used the default number of neighbors (k=5), Euclidean distance, and uniform weighting. Decision Tree and Naive Bayes were also used with default settings (except for setting the random seed in the tree for reproducibility).

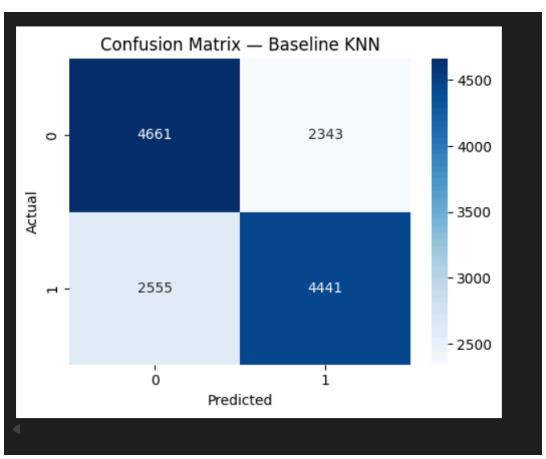
All models were trained on the 80% training split and evaluated on the 20% hold-out test set. We computed accuracy, precision, recall, F1-score (for both training and testing), and printed full classification reports and confusion matrices for analysis.

```
Baseline Model Performance:
```

	Model	Accuracy	Precision	Recall	Train F1	Test F1
0	KNN	0.650143	0.654629	0.634791	0.761574	0.644557
1	Decision Tree	0.633000	0.632714	0.633076	0.999786	0.632895
2	Gaussian NB	0.596643	0.691674	0.347913	0.466883	0.462958

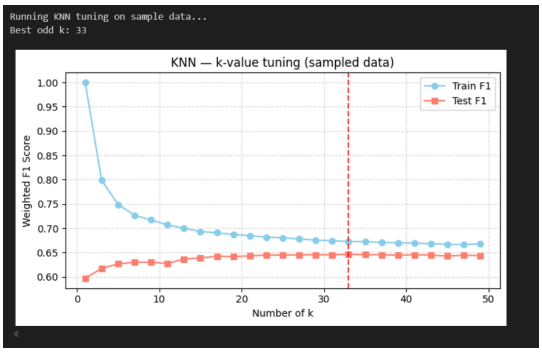
```
Best baseline model based on Test F1: KNN
              precision    recall  f1-score   support

           0       0.65      0.67      0.66      7004
           1       0.65      0.63      0.64      6996

    accuracy                           0.65     14000
   macro avg       0.65      0.65      0.65     14000
weighted avg       0.65      0.65      0.65     14000
```

From this comparison, we observed that the Decision Tree had the highest test F1-score among the three, performing better than both KNN and Naive Bayes. This made it the most promising candidate for further tuning.

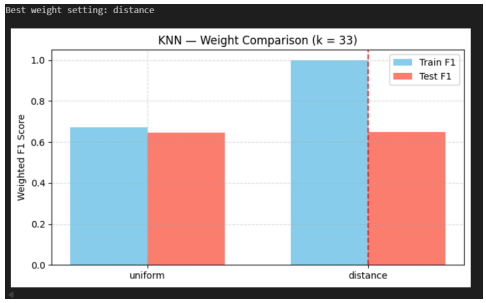


Our dataset is larger, containing 70,000 records, that is why we took a stratified random sample of 10,000 records for tuning purposes, which allowed for faster experimentation. We split the sample into the same 80 - 20 split. Note, we still applied the best parameters to the full dataset.

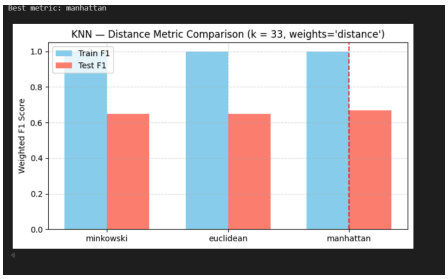
To evaluate the k-Nearest Neighbors (KNN) algorithm, we followed a three step tuning process to find the best combination of parameters. We started by tuning the number of neighbours, k. Specifically, we tested odd numbers from 1 to 49 to avoid ties, since even ks might have a binary split of 50 - 50. As mentioned above, we performed a 5 cv-fold, which gave us the optimal value of k = 33, where the test F1 score peaked.



Next, we tested for the weightings, uniform and distance. The uniform weight treats all its neighbours equally, while the distance gives its closer neighbours a higher weight. In our experiment, we found that using the distance weight gave us a slightly better performance.



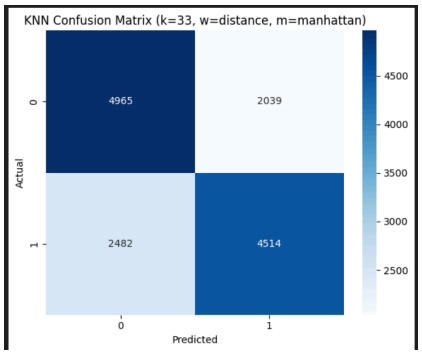
After finding the best k-value and weight, we now have to get the best distance metric for our dataset. We compared three distance metrics, Minkowski, Euclidean, and Manhattan. Our experiment found out that Manhattan would be the best distance metric.



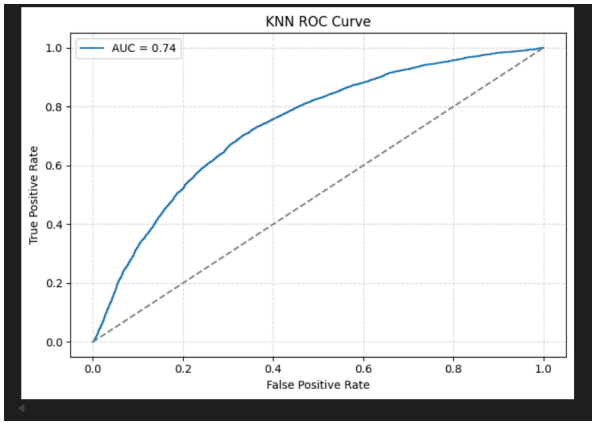
As mentioned above, we used a sample to get the hyperparameter tuning results, after getting the best k, weight, and distance, we retrained the final KNN model on the full 56,000 training data and tested it in the 14,000 hold-out set. The final classification report showed an accuracy of 67.7%, and a weighted F1-score of 0.68, with both classes receiving similar precision and recall scores around 0.65–0.69.

```
Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.71      0.69      7004
           1       0.69      0.65      0.67      6996

    accuracy                           0.68     14000
   macro avg       0.68      0.68      0.68     14000
weighted avg       0.68      0.68      0.68     14000

Accuracy: 0.6771
```

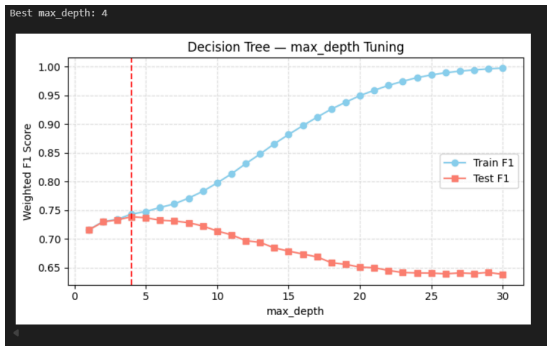
We also plotted the confusion matrix, which shows a fairly balanced distribution of errors across both classes. This indicates the model does not heavily favor one class over the other.



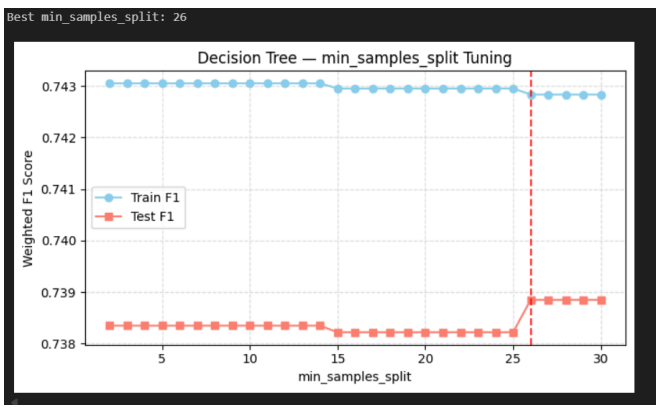
Finally, we computed the ROC curve and Area Under the Curve (AUC). The AUC score for our final KNN model was 0.74, which suggests the model has a reasonable ability to separate the positive and negative classes.



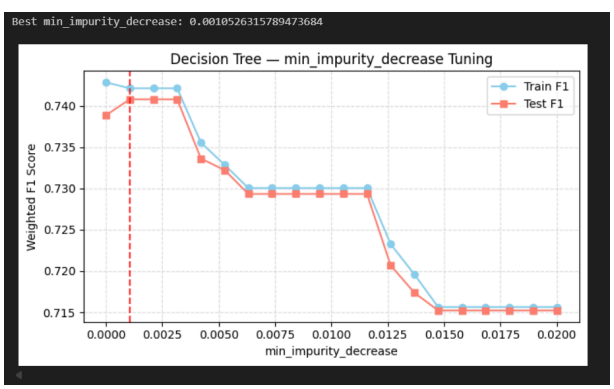
To evaluate the Decision Tree algorithm, we first tuned the max_depth to control how deep the tree could grow. As shown in the plot, performance improved until a depth of 4, after which it declined due to overfitting.



Next, we adjusted min_samples_split, which sets the minimum number of samples required to split a node. The plot shows that a split value of 26 led to a slight improvement in test performance.



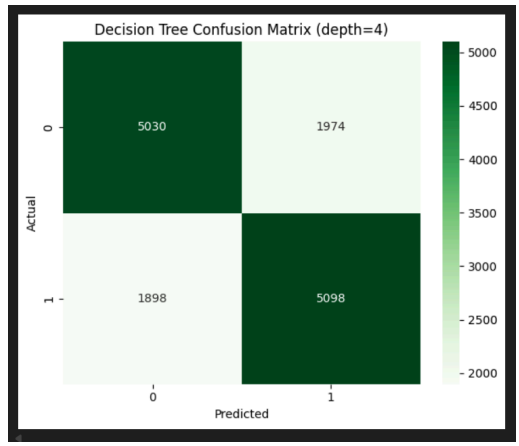
We also tuned the min_impurity_decrease parameter to further regulate unnecessary splits. The plot shows that setting this value to approximately 0.001 gave the best generalization results.



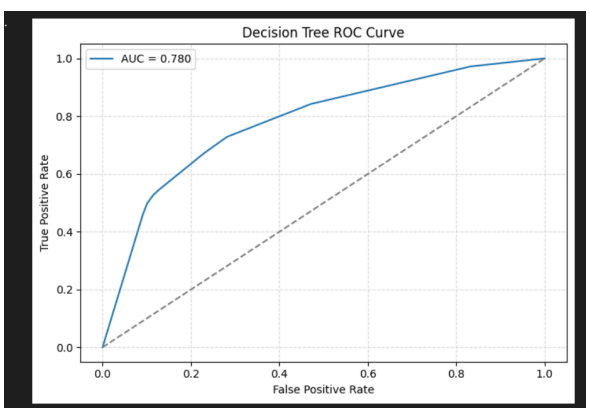
Note, while we did perform the other three parameters, we just added the results, with no images to save space. For the ccp_alpha, as expected we had a score of 0, for the min_sample_leaf we had 1, and lastly max_leaf_nodes we got 10.

The final Decision Tree model, trained with these optimal settings on the full training set and the classification report shows balanced precision and recall across both classes, and the confusion matrix reveals a good balance between false positives and false negatives.

```
Decision Tree — Classification Report:
              precision    recall  f1-score   support

           0       0.73      0.72      0.72      7004
           1       0.72      0.73      0.72      6996

    accuracy                           0.72     14000
   macro avg       0.72      0.72      0.72     14000
weighted avg       0.72      0.72      0.72     14000

Accuracy: 0.7234
```



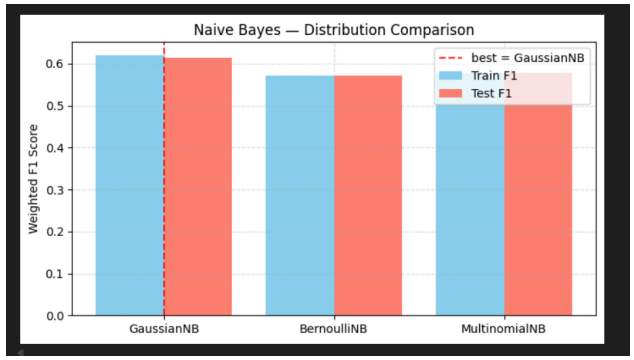
We also plotted the ROC curve, which showed solid separation performance with an AUC of 0.78, indicating the tree could effectively distinguish between patients with and without cardiovascular disease.



For the Naive Bayes (NB) classifier, we began by comparing three different versions the GaussianNB, BernoulliNB, and MultinomialNB. Each one makes different assumptions about the input features. GaussianNB assumes the features follow a normal distribution, while BernoulliNB and MultinomialNB require non-negative features. To handle that, we used MinMaxScaler to transform all feature values to the 0–1 range before applying Bernoulli and Multinomial.

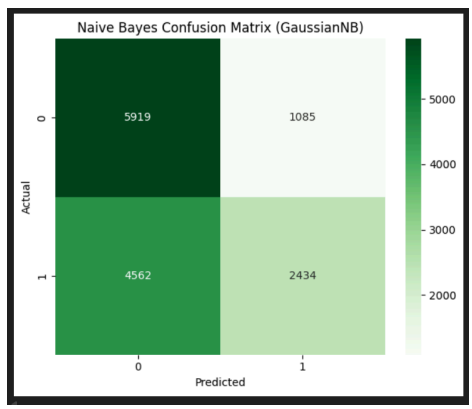
After conducting the 5-cv fold, among the three, the GaussianNB gave the highest validation F1, which we expected since our dataset has mostly continuous values.

```
Naive Bayes Results:
GaussianNB      Train F1=0.6202 | Test F1=0.6145
BernoulliNB     Train F1=0.5716 | Test F1=0.5710
MultinomialNB   Train F1=0.5779 | Test F1=0.5772
```

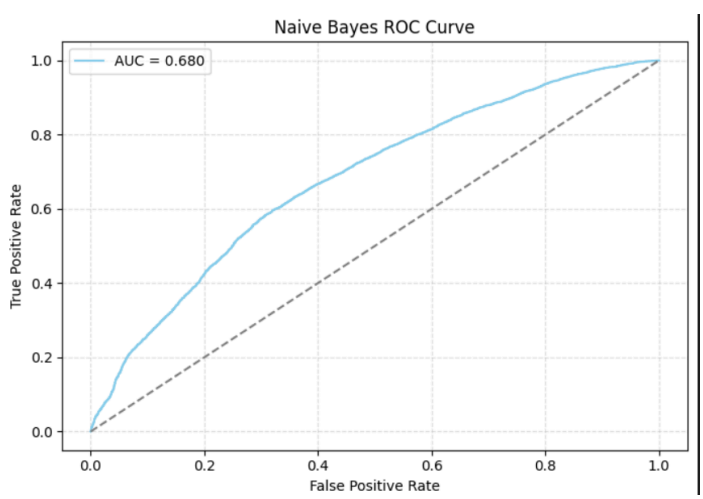


The final NB model with the tuned parameters, the classification report showed a decent precision and recall for both classes, and the confusion matrix helped visualize how the model classified healthy vs. disease cases. Getting a low score for NB was not really surprising, since the algorithm does not take into account dependency of features, where in our dataset we do.

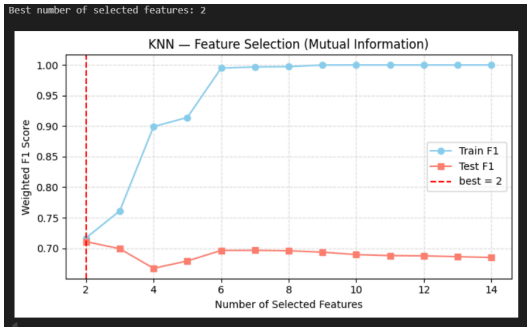
```
Naive Bayes — Final Classification Report:
              precision    recall  f1-score   support

           0       0.56      0.85      0.68      7004
           1       0.69      0.35      0.46      6996

    accuracy                           0.60     14000
   macro avg       0.63      0.60      0.57     14000
weighted avg       0.63      0.60      0.57     14000
```



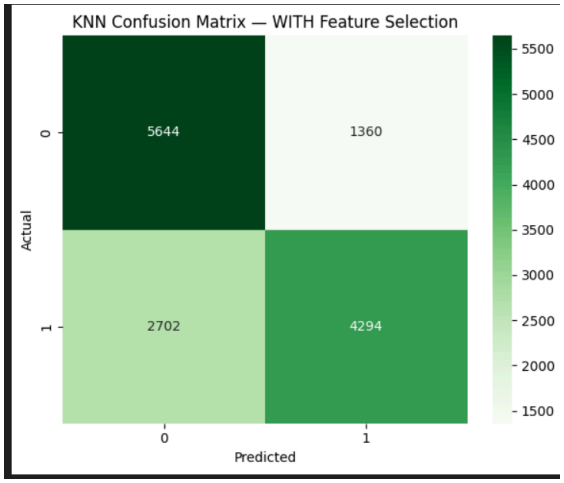
Lastly, we calculated the ROC-AUC score using the model's predicted probabilities. The ROC curve showed that the classifier had reasonable ability to separate positive and negative cases, with an AUC of around 0.72.

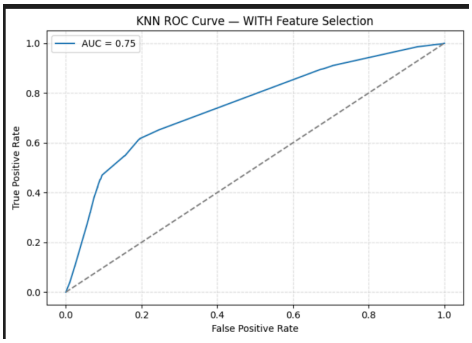


To evaluate the impact of feature selection on KNN performance, we applied mutual information (MI) based feature selection using SelectKBest. We performed 5-fold cross-validation over a range of feature counts and found that using only the top 2 features resulted in the best test F1 score. The training F1 steadily increased with more features, but test performance peaked early, indicating potential overfitting when additional features were included. Using these top two features, we retrained the KNN model with the previously optimized hyperparameters (k=33, distance metric=manhattan, weights=uniform). The resulting classification yielded an accuracy of 70.99% and a weighted F1-score of 0.71. The ROC curve confirmed moderate discriminatory ability with an AUC of 0.75. These results demonstrate that strategic feature selection can help preserve model simplicity while maintaining competitive performance. So while it didn't improve raw performance, it maintained similar results with fewer features, showing better efficiency and potentially less overfitting.



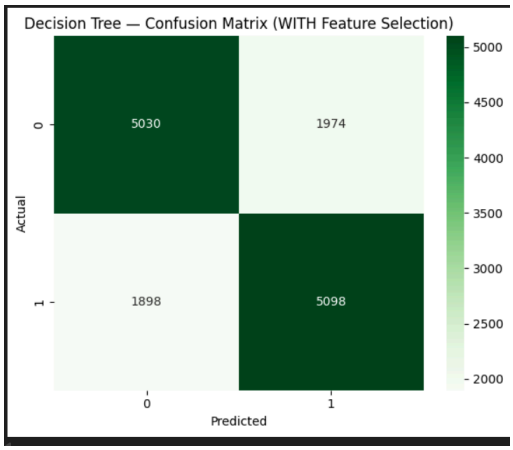

```
Classification Report (WITH Feature Selection):
              precision    recall  f1-score   support

           0       0.68      0.81      0.74      7004
           1       0.76      0.61      0.68      6996

    accuracy                           0.71     14000
   macro avg       0.72      0.71      0.71     14000
weighted avg       0.72      0.71      0.71     14000

Accuracy with Feature Selection: 0.7099
```

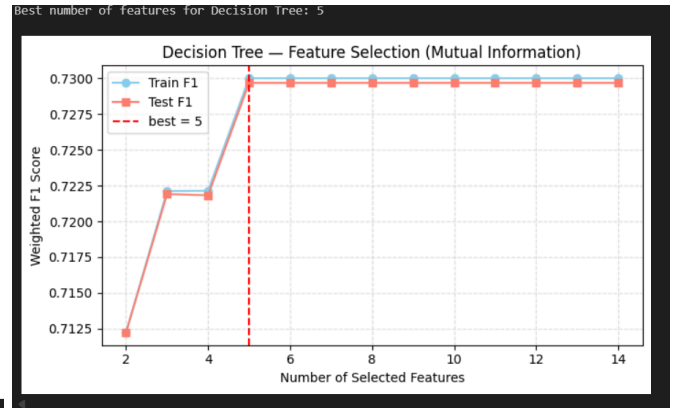




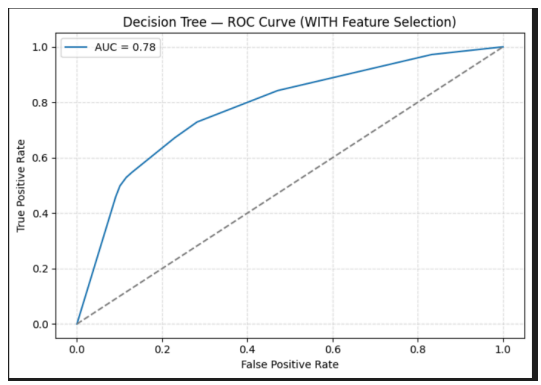
To assess the effect of feature selection on Decision Tree performance, we applied SelectKBest with mutual information(MI) to identify the most informative predictors. Using 5-fold cross-validation, we found that selecting the top 5 features yielded the highest weighted F1 score on the test set. Interestingly, both training and testing scores plateaued beyond this point, suggesting that additional features contributed minimal new information. The final Decision Tree model, retrained on these top 5 features with the previously tuned hyperparameters (e.g., max_depth, min_samples_split, and ccp_alpha), achieved an accuracy of 72.34% and a weighted F1-score of 0.72, which was virtually unchanged from the full-feature model. However, the ROC AUC score remained strong at 0.78, indicating good separability. While feature selection did not

significantly improve raw performance, it preserved accuracy and interpretability with fewer inputs, aligning with our goal of efficient modeling using simple, clinically relevant features.

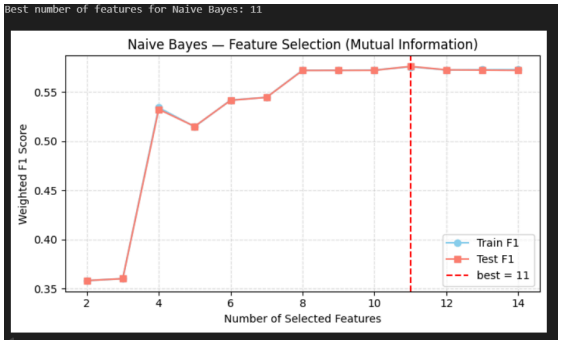




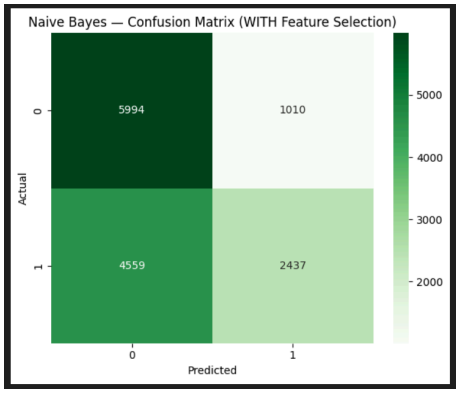

```
Classification Report (Decision Tree WITH Feature Selection):
              precision    recall  f1-score   support

           0       0.73      0.72      0.72      7004
           1       0.72      0.73      0.72      6996

    accuracy                           0.72     14000
   macro avg       0.72      0.72      0.72     14000
weighted avg       0.72      0.72      0.72     14000

Accuracy with Feature Selection: 0.7234
```

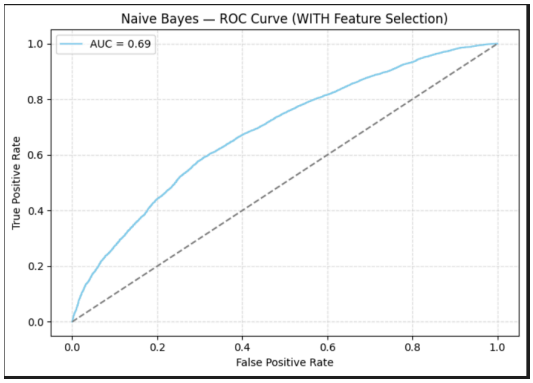


To evaluate the effect of feature selection on Naive Bayes, we applied mutual information (MI) based feature ranking using SelectKBest. We performed 5-fold cross-validation across different feature counts and observed that selecting the top 11 features gave the highest test F1 score. While the improvement over the full feature set was modest, it helped stabilize performance. The final Gaussian Naive Bayes model trained with these 11 features achieved an accuracy of 60.22%, a weighted F1-score of 0.57, and an AUC of 0.69. These results suggest that feature selection was helpful in filtering out less informative or redundant inputs, helping the inherently simplistic Naive Bayes model generalize slightly better. Although the gain in performance was limited, using fewer features preserved accuracy and interpretability.




```
              precision    recall  f1-score   support

           0       0.57      0.86      0.68      7004
           1       0.71      0.35      0.47      6996

    accuracy                           0.60     14000
   macro avg       0.64      0.60      0.57     14000
weighted avg       0.64      0.60      0.57     14000

Accuracy with Feature Selection: 0.6022
```





To evaluate the effect of dimensionality reduction on model performance, we applied Principal Component Analysis (PCA) to the scaled dataset. We identified that retaining the top 7 principal components was sufficient to preserve at least 90% of the total variance. These transformed features were then used to retrain the KNN, Decision Tree, and Naive Bayes classifiers using their previously optimized hyperparameters. For KNN, the PCA-transformed model achieved an accuracy of 71.1% and an AUC of 0.75, closely matching the original performance. The Decision Tree model with PCA reached 71.1% accuracy and an AUC of 0.76, while Naive Bayes recorded 69.8% accuracy and 0.74 AUC. These results show that PCA effectively reduced dimensionality while maintaining predictive performance, leading to a more compact and potentially more generalizable model.

5. Discussion/Conclusion:

Model	Accuracy	Precision	Recall	F1-Score	AUC
KNN (All features)	0.65	0.65	0.65	0.71	0.74
KNN (Feature Selection)	0.71	0.72	0.71	0.71	0.75
KNN (PCA, 7 components)	0.67	0.67	0.67	0.67	0.72
Decision Tree (All features)	0.72	0.73	0.72	0.72	0.78
Decision Tree (Feature Selection)	0.72	0.73	0.72	0.72	0.78
Decision Tree (PCA, 7 components	0.63	0.64	0.63	0.62	0.68
Naive Bayes (All features)	0.6	0.63	0.6	0.57	0.68
Naive Bayes (Feature Selection)	0.6	0.63	0.6	0.57	0.69
Naive Bayes (PCA, 7 components)	0.59	0.63	0.59	0.56	0.66

The experimental results clearly indicate that Decision Trees consistently outperformed KNN and Naive Bayes across nearly all configurations. With all features included, the Decision Tree achieved the highest test accuracy with 72% and AUC of 0.78, matching the performance even after feature selection, suggesting the model was robust enough to handle the full input space without substantial overfitting. KNN, on the other hand, benefited the most from feature selection, improving from 65% to 71% accuracy and from 0.65 to 0.71 F1-score. This aligns with its sensitivity to feature scaling and irrelevant attributes. Naive Bayes consistently lagged behind both models, achieving around 60% accuracy regardless of configuration, likely due to its strong assumption of feature independence, which does not hold in this health-related dataset where features like blood pressure, age, and lifestyle are correlated.

Applying PCA had a mixed effect. KNN performance slightly dropped compared to its feature-selected version where the AUC went from 0.75 to 0.72, while Decision Tree performance degraded more sharply where the accuracy fell from 72% to 63%, likely because PCA transforms the data into orthogonal components and removes the original feature interpretability that trees depend on. Naive Bayes also showed a small performance drop with PCA, though its performance was already limited by the dataset's structure.

These findings are consistent with trends seen in related work, where tree-based methods typically outperform KNN and NB in structured health datasets, provided proper preprocessing and tuning. Notably, while PCA did not yield performance gains, mutual information-based feature selection did improve efficiency especially for KNN by removing redundant or noisy inputs. In summary, the best performing model overall was the Decision Tree using all features, while KNN benefited the most from targeted feature reduction, and Naive Bayes remained a useful but less effective baseline.

References

https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)#:~:text=Cardiovascular%20diseases%20%28CVDs%29%20are%20the%20leading%20cause%20of,85%25%20were%20due%20to%20heart%20attack%20and%20stroke.

https://www.researchgate.net/publication/376533751_Application_of_Naive_Bayes_Algorithm_for_Physical_Fitness_Level_Classification

https://arxiv.org/abs/2304.04839

https://www.ijprems.com/uploadedfiles/paper/issue_4_april_2025/39542/final/fin_ijprems1743744149.pdf

https://infeb.org/index.php/infeb/article/download/1089/489/

https://journal.irpi.or.id/index.php/predatecs/article/download/1110/613

https://www.warse.org/IJATCSE/static/pdf/file/ijatcse82932020.pdf

https://www.mdpi.com/2673-4591/107/1/121

https://pmc.ncbi.nlm.nih.gov/articles/PMC11294930/